

02 — Alignment e Padding

A surpresa

```
struct A {  
    char x;    // 1 byte  
    int y;     // 4 bytes  
};  
  
sizeof(A);    // 8, não 5
```

Três bytes apareceram do nada.

Alignment (alinhamento)

O hardware lê memória de forma eficiente quando um valor está num endereço que é múltiplo do seu tamanho. Um `int` de 4 bytes quer um endereço divisível por 4; um `double` de 8 bytes quer um divisível por 8. Acesso desalinhado é mais lento, e em algumas arquiteturas trava de vez.

Então o compilador insere **padding** — bytes não usados — pra manter cada campo alinhado.

```
struct A { char x; int y; };  
  
offset  conteúdo  
0      x          (1 byte)  
1      PADDING  
2      PADDING  
3      PADDING  
4      y          (4 bytes, agora num múltiplo de 4)  
...  
      total: 8
```

As regras

1. Cada campo é colocado no próximo offset que é múltiplo do seu alinhamento.
2. O alinhamento do próprio struct é o do membro mais exigente em alinhamento.
3. O tamanho total é arredondado pra cima até um múltiplo do alinhamento do struct, pra que arrays dele continuem alinhados.

A regra 3 é a que as pessoas esquecem, e é o motivo do padding no final existir:

```
struct B {  
    int y;    // 4 bytes no offset 0  
    char x;   // 1 byte no offset 4  
};           // total 8: três bytes de padding NO FINAL
```

Sem isso, `B array[2]` colocaria o segundo elemento no offset 5 e desalinharía o `int` dele.

A ordem dos campos importa

```
struct Bad { char a; int b; char c; }; // 12 bytes
struct Good { int b; char a; char c; }; // 8 bytes
```

Bad:	Good:
0 a	0 b
1 PAD	1 b
2 PAD	2 b
3 PAD	3 b
4 b	4 a
5 b	5 c
6 b	6 PAD
7 b	7 PAD
8 c	total 8
9 PAD	
10 PAD	
11 PAD	
total 12	

Mesmo dado, 50% mais memória. **Ordenar campos do maior pro menor é uma heurística confiável.**

Para um milhão de elementos isso é 12 MB contra 8 MB — e pra um kernel memory-bound, 4 MB de padding são 4 MB de banda de memória desperdiçada.

Inspecionando

```
#include <cstddef>

sizeof(A);           // tamanho total
alignof(A);          // requisito de alinhamento
offsetof(A, y);      // offset em bytes de um campo
```

`offsetof` é a forma direta de ver pra onde o padding foi. O exercício 02 usa isso.

Controlando

```
struct alignas(16) Aligned { float x, y, z, w; }; // força alinhamento de 16 bytes
```

`#pragma pack(1)` remove o padding completamente, produzindo um struct menor e mais lento. Ocasionalmente correto para formatos de arquivo ou protocolos de rede; quase nunca correto para compute.

`alignas` é o que importa aqui — alinhamento de 16 bytes é o que torna loads vetorizados válidos.

Por que isso importa para CUDA

- **Padding desperdiça banda de memória.** Para um kernel memory-bound, bytes movidos é o custo, e bytes de padding são movidos à toa.
- **Alignment habilita loads vetorizados.** `float4` tem 16 bytes alinhados a 16 bytes, e ler um é uma única instrução de 16 bytes. Isso exige que o endereço esteja devidamente alinhado; dado desalinhado silenciosamente cai pro caminho lento.
- **CUDA tem tipos vetoriais alinhados embutidos** — `float2`, `float4`, `int4` — que existem exatamente por isso. As declarações `alignas` deles são o mecanismo.
- **Acesso desalinhado à memória global degrada o coalescing.** Um warp lendo 128 bytes começando num endereço desalinhado toca duas regiões de memória em vez de uma.

`experiments/memory-access/` neste repositório vai medir o ganho de load vetorizado. Este arquivo é onde o motivo disso está explicado.